58

BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 4: HikariCP FastPath Bağlantı Havuzu Mekaniği (ConcurrentBag & ThreadLocal Handoff)

MAANG Case 4: HikariCP FastPath Connection Pool (ConcurrentBag Architecture)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                HIKARICP CONCURRENTBAG VE FASTPATH MİMARİSİ    │
└──────────────────────────────────────────────────────────────┘

 Thread Bağlantı İstediğinde (getConnection()):

 1. FASTPATH (SIFIR KİLİT!):
    ThreadLocal önbelleğine bak ──► [ Daha önce kullandığım boş bağlantı var mı? ]
    ──EVET──► CAS ile durumunu "IN_USE" yap ve DÖN! (Mikrosaniye altı!)

 2. SHARED SCAN (PAYLAŞIMLI TARAMA):
    ──HAYIR──► CopyOnWriteArrayList paylaşımlı listesini tara!
               Boş bir bağlantı bulursan CAS ile kap!

 3. THREAD HANDOFF (EL DEĞİŞTİRME):
    ──HAYIR──► SynchronousQueue benzeri Handoff kuyruğunda bekle!
               Başka bir thread bağlantıyı bıraktığı an elden ele teslim edilir!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

HikariCP dünyanın en hızlı veritabanı bağlantı havuzudur. Eski havuzlar tek bir kapıcı koyar; 100 kişi o kapıcının önünde sıraya girerdi (`synchronized`). HikariCP ise şöyle der: 'Dün kullandığın anahtarı kendi cebine (`ThreadLocal`) koy. Yarın tekrar gelirsen kapıcıya hiç sormadan cebindeki anahtarı çıkarıp doğrudan içeri gir!' Cebinde anahtar yoksa masaya bakar, masada da yoksa anahtarı getirecek ilk kişiyi el sıkışma noktasında bekler!

🇬🇧 ENGLISH (ELI5)

HikariCP is universally recognized as the fastest JDBC connection pool on earth. Legacy pools (e.g. DBCP, c3p0) guarded connections behind a single synchronized gate. HikariCP implemented `ConcurrentBag`: each thread caches its most recently used connection in a `ThreadLocal`. When requesting a connection, it inspects its private thread cache first—if idle, it claims it with zero lock contention! If absent, it scans the shared bag; if empty, it registers on a zero-capacity handoff queue.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HikariCP ConcurrentBag'in 3 Katmanlı Çalma Hiyerarşisi: 1) Katman 1 (ThreadLocal FastPath): `threadLocalBagEntry.get()` çağrılır. Eğer referans varsa ve durumu `STATE_NOT_IN_USE` ise, tek bir atomik `CAS(STATE_NOT_IN_USE -> STATE_IN_USE)` ile kilit almadan bağlantı tahsis edilir. CPU context switch ve kilit çekişmesi sıfırdır. 2) Katman 2 (Shared List Scanning): ThreadLocal boşsa veya başka bir thread tarafından çalınmışsa, havuz `CopyOnWriteArrayList` üzerinde dolaşarak uygun boş bağlantıları CAS ile arar. 3) Katman 3 (Synchronous Handoff): Havuz tamamen meşgulse çağıran thread `SynchronousQueue` benzeri adil bir teslim kuyruğunda bekletilir. Başka bir thread `close()` çağırdığında bağlantıyı havuza iade etmek yerine doğrudan bekleyen thread'e teslim eder (Thread-to-Thread Handoff).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HikariCP ConcurrentBag Triple-Tiered Resolution: 1) Tier 1 (ThreadLocal FastPath): Queries thread-local cache. If a previously utilized entry is present in state `STATE_NOT_IN_USE`, an atomic CAS transitions state to `STATE_IN_USE`. Total time: nanoseconds, zero lock contention. 2) Tier 2 (Shared Bag Sweep): If the ThreadLocal is vacant or stolen, the thread iterates over a lock-free `CopyOnWriteArrayList` of pool entries, attempting CAS acquisitions. 3) Tier 3 (Synchronous Handoff Queuing): If all connections are occupied, the thread parks on a specialized `SynchronousQueue` handoff. When a loaner thread invokes `close()`, instead of recycling the entry to the shared list, it hands the connection off directly to the waiting worker thread.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: HikariCP ConcurrentBag FastPath Mantığının Minyatür Modeli
import java.util.concurrent.atomic.AtomicInteger;

public class MiniConcurrentBagPool {
    private static final int STATE_NOT_IN_USE = 0;
    private static final int STATE_IN_USE = 1;

    public static class BagEntry {
        final String connectionId;
        final AtomicInteger state = new AtomicInteger(STATE_NOT_IN_USE);
        public BagEntry(String id) { this.connectionId = id; }
    }

    // Her thread'in son kullandığı bağlantıyı tutan FastPath önbelleği
    private final ThreadLocal<BagEntry> fastPathCache = new ThreadLocal<>();
    private final BagEntry[] sharedBag;

    public MiniConcurrentBagPool(int size) {
        sharedBag = new BagEntry[size];
        for (int i = 0; i < size; i++) {
            sharedBag[i] = new BagEntry("CONN_JDBC_" + i);
        }
    }

    public BagEntry borrowConnection() {
        // 1. ADIM: FastPath (ThreadLocal'dan kilit-siz al!)
        BagEntry cached = fastPathCache.get();
        if (cached != null && cached.state.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
            System.out.println("⚡ FastPath ile anında alındı: " + cached.connectionId);
            return cached;
        }

        // 2. ADIM: Paylaşımlı havuzu tara
        for (BagEntry entry : sharedBag) {
            if (entry.state.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
                fastPathCache.set(entry); // Gelecek sefer için önbelleğe al!
                System.out.println("🔍 Shared Bag'den bulundu: " + entry.connectionId);
                return entry;
            }
        }
        throw new RuntimeException("Tüm bağlantılar dolu (Havuz Tükendi)!");
    }

    public void releaseConnection(BagEntry entry) {
        entry.state.set(STATE_NOT_IN_USE);
    }

    void main() {
        var pool = new MiniConcurrentBagPool(5);
        var c1 = pool.borrowConnection();
        pool.releaseConnection(c1);
        // İkinci istek FastPath üzerinden yıldırım hızında gelir!
        var c2 = pool.borrowConnection();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

HikariCP'de havuz boyutunu 1000 yapmak bağlantı hızını ARTIRMAZ, aksine veritabanı disk I/O ve CPU'sunu context switch ile boğar! HikariCP formülü: `pool_size = Tn × (CoreCount × 2 + EffectiveSpindleCount)` şeklindedir. Çoğu MAANG servisi için 10-20 bağlantı binlerce eşzamanlı isteği karşılamaya yeterlidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Configuring pool sizes to 500+ does NOT improve throughput; it cripples database CPU and disk subsystems with context-switching thrash. Follow the HikariCP sizing formula: `pool_size = CoreCount × 2 + SpindleCount`. For typical services, a modest pool of 10-20 connections out-scales bloated pools by 10x.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• HikariCP'nin sırrı ConcurrentBag'dir: ThreadLocal FastPath ile önce kendi önbelleğine bakar, yoksa paylaşımlı havuzdan CAS ile çalar.

🎯 Key Takeaways for the Interview

• HikariCP's supremacy lies in ConcurrentBag: ThreadLocal FastPath bypasses locks; shared sweeps and direct handoffs eliminate pool queuing.